



Lab 9 - Implementing New CCA

Agenda

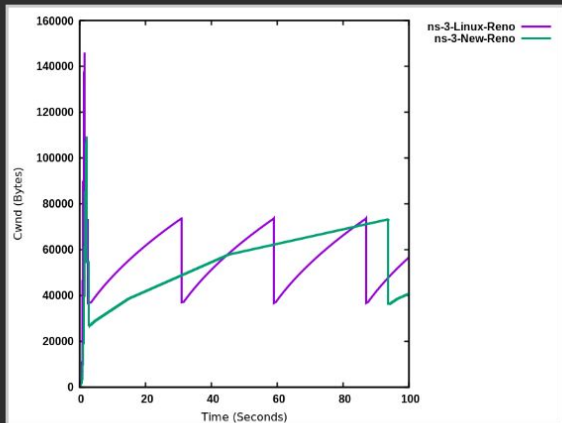


- Tracing Congestion Window, RTT Changes with the help of ns3 callback functions.
- Creating a custom TCP Congestion Control algorithm and testing it out in ns3.
- Understanding Jain's fairness index.

Diving deeper into TCP

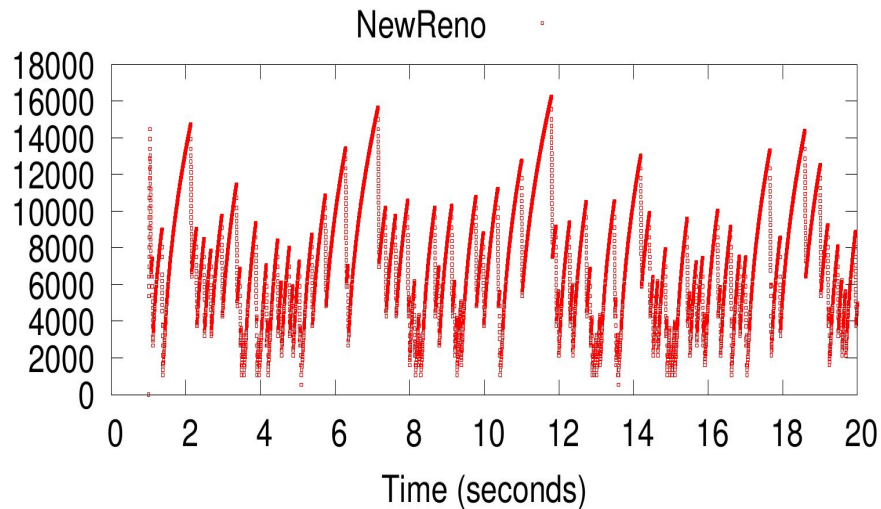
- Observing Congestion Window Changes

Following graphs shows the behavior of window growth in TCP Linux **Reno** and TCP New**Reno** with delayed acknowledgement of 2 segments:



ns-3 TCP New**Reno** vs. ns-3 TCP Linux **Reno**

cwnd (number of segments)



Callback functions (Demo_2.cc)



- Mainly used for tracking changes in network metrics.
- `Ptr<Socket> ns3TcpSocket = Socket::CreateSocket (nodes.Get (0), TcpSocketFactory::GetTypeId ());`
- A TCP socket is created on Node 0, which is used for the TCP connection in the simulation. This socket will be monitored for changes in its attributes, like congestion window size.

Callback functions (Demo_2.cc)



- The **TraceConnectWithoutContext** method is used to connect the **CongestionWindow** trace source of the **ns3TcpSocket**.
- **MakeBoundCallback** binds the **CwndChange** function to the trace source. This function will be called whenever the congestion window changes.
- The **CwndChange** function takes an **OutputStreamWrapper** (used for writing to a file), the old congestion window size, and the new size. It logs the changes to the file "**seventh.cwnd**".

Some fundamentals ...



- The file paths for some fundamental classes or files are as follows:
 - /home/arindam/Desktop/Arindam Thesis/Simulation Setup/ns-allinone-3.44/ns-3.44/src/internet/model/[tcp-socket-base.cc](#)
 - /home/arindam/Desktop/Arindam Thesis/Simulation Setup/ns-allinone-3.44/ns-3.44/src/internet/model/tcp-congestion-ops.cc
- All operations that are delegated to a congestion control are contained in the class TcpCongestionOps. It mimics the structure tcp_congestion_ops of Linux, and the following operations are defined:

```
virtual std::string GetName() const;
virtual uint32_t GetSsThresh(Ptr<const TcpSocketState> tcb, uint32_t bytesInFlight);
virtual void IncreaseWindow(Ptr<TcpSocketState> tcb, uint32_t segmentsAacked);
virtual void PktsAacked(Ptr<TcpSocketState> tcb, uint32_t segmentsAacked, const Time& rtt);
virtual Ptr<TcpCongestionOps> Fork();
virtual void CwndEvent(Ptr<TcpSocketState> tcb, const TcpSocketState::TcpCaEvent_t event);
```

Some fundamentals ...



- The most interesting methods to write are **GetSsThresh** and **IncreaseWindow**. The latter is called when **TcpSocketBase** decides that it is time to increase the congestion window. Much information is available in the Transmission Control Block, and the method should increase `cWnd` and/or `ssThresh` based on the number of segments acked.
- **GetSsThresh** is called whenever the socket needs an updated value of the slow start threshold. This happens after a loss; congestion control algorithms are then asked to lower such value, and to return it.

Some fundamentals ...



- **PktsAked** is used in case the algorithm needs timing information (such as RTT), and it is called each time an ACK is received.
- **CwndEvent** is used in case the algorithm needs the state of socket during different congestion window event.

Some fundamentals ...



- All operations that are delegated to a loss recovery are contained in the class `TcpRecoveryOps` and are given below:

```
virtual std::string GetName() const;
virtual void EnterRecovery(Ptr<const TcpSocketState> tcb, uint32_t unAckDataCount,
                          bool isSackEnabled, uint32_t dupAckCount,
                          uint32_t bytesInFlight, uint32_t lastDeliveredBytes);
virtual void DoRecovery(Ptr<const TcpSocketState> tcb, uint32_t unAckDataCount,
                       bool isSackEnabled, uint32_t dupAckCount,
                       uint32_t bytesInFlight, uint32_t lastDeliveredBytes);
virtual void ExitRecovery(Ptr<TcpSocketState> tcb, uint32_t bytesInFlight);
virtual void UpdateBytesSent(uint32_t bytesSent);
virtual Ptr<TcpRecoveryOps> Fork();
```

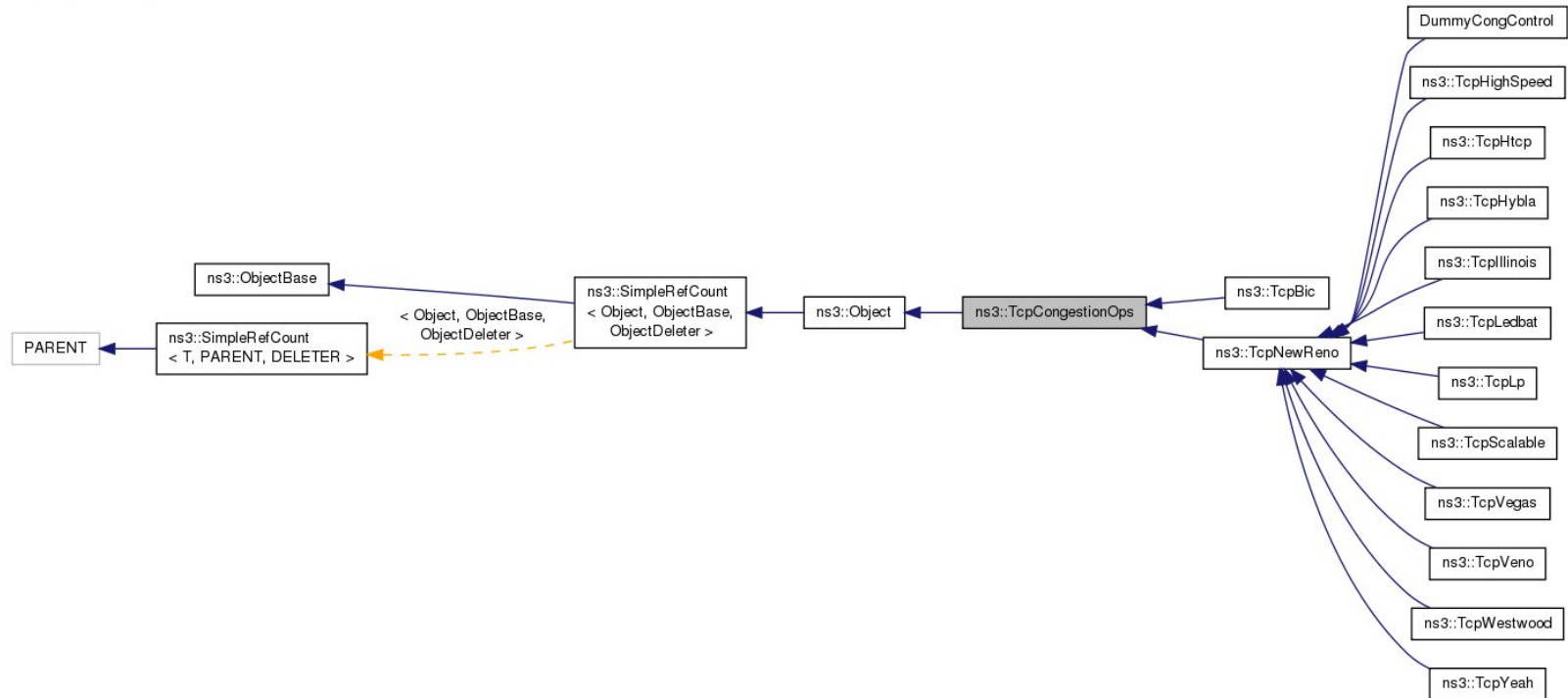
Creating a New Congestion Control Algorithm



- Navigate to the `src/internet/model/` directory in your ns-3 installation. Create a new header (`.h`) and implementation (`.cc`) file for your custom congestion control algorithm. For example, you might name your files `tcp-custom.cc` and `tcp-custom.h`.
- In the header file (`tcp-custom.h`), define your new congestion control class by inheriting from `TcpCongestionOps` or if you wish you can even set your parent as `TcpSocketBase`

Inheritance Diagram for TcpCongestionOps

▼ Inheritance diagram for ns3::TcpCongestionOps:



Creating a New Congestion Control Algorithm



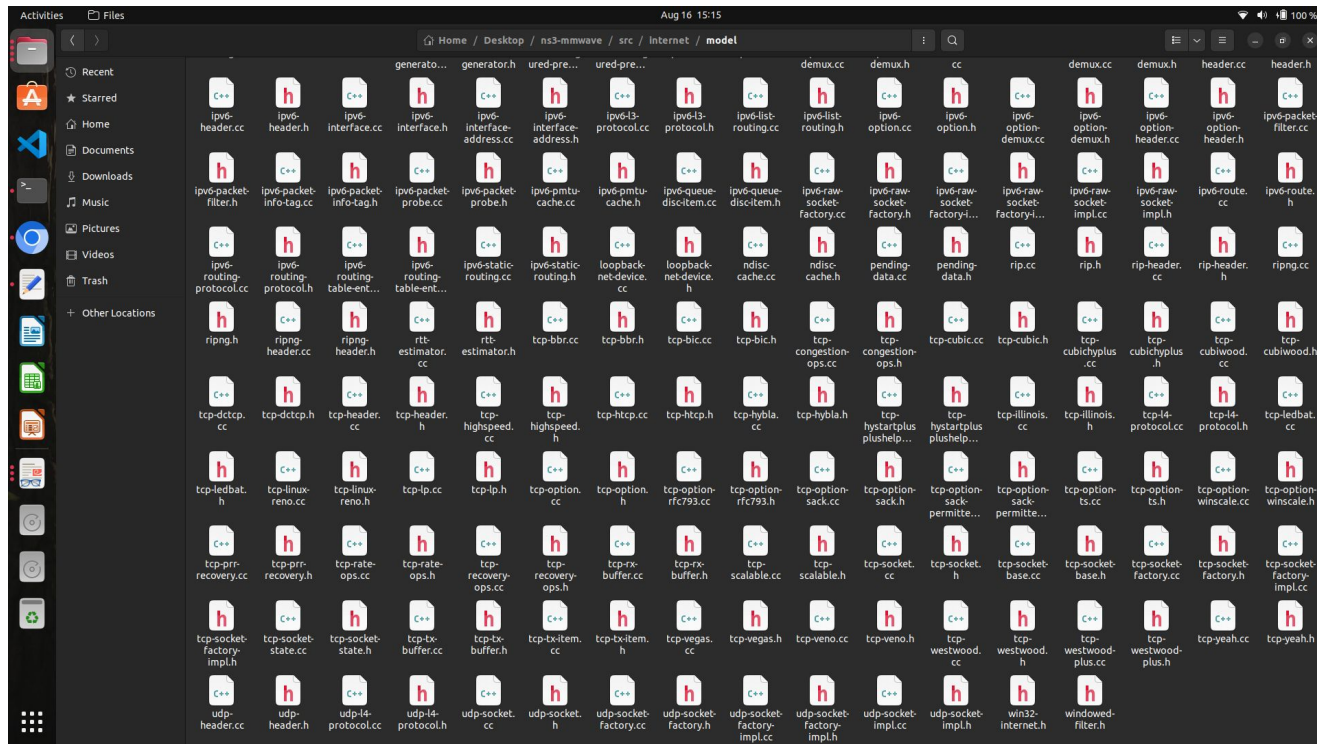
- Navigate to `/home/arindam/Desktop/Arindam Thesis/Simulation Setup/ns-allinone-3.44/ns-3.44/src/internet/CMakeLists.txt` and add the new congestion control algorithm to it under model and helper.
- Navigate to `/home/arindam/Desktop/Arindam Thesis/Simulation Setup/ns-allinone-3.44/ns-3.44/build/include/ns3` and create a `tcp-newalgo.h` file and write (needed if using ns-3-dev):
 - `/home/arindam/Desktop/Arindam Thesis/Simulation Setup/ns-allinone-3.44/ns-3.44/src/internet/model/tcp-newalgo.h`, which is essentially the path to the header file of the newly created congestion control algorithm

Creating a New Congestion Control Algorithm

- Adding the newly created congestion control algorithm's source code and header files to CMakeLists.txt

```
model/tcp-cubic.cc  
model/tcp-dummy.cc  
model/tcp-dctcp.cc  
model/tcp-header.cc  
model/tcp-highspeed.cc  
model/tcp-htcp.cc
```

```
model/tcp-cubic.h  
model/tcp-dummy.h  
model/tcp-dctcp.h  
model/tcp-header.h  
model/tcp-highspeed.h  
model/tcp-htcp.h  
model/tcp-hybla.h
```



Jain's Fairness Index

- Jain's Index

= Square of the sum of all throughputs/ (N x sum of squares of all throughputs).

= Square of average throughput/ Average of squared throughput.

- Here N is the total number of traffic flows.

- The formula:

$$\frac{\left[\sum_{i=1}^n x_i \right]^2}{n \sum_{i=1}^n x_i^2}$$

Jain's Fairness Index



- If there are n TCP flows in the network, then their throughput is denoted as $x_1, x_2, \dots, x_n$. We know that, throughput is the rate of successful packets/message delivery over an established network connection.
- If the $J=1$, then the TCP under experiment is perfectly fair with other TCP flows. It can be implied that TCP does not hurt other TCP flows in the network. Any new TCP algorithm developed today has to prove its efficiency in terms of TCP fairness measure

References



- <https://github.com/nsnam/ns-3-dev-git/blob/master/src/internet/doc/tcp.rst>
- https://www.nsnam.org/doxygen/dc/d64/classns3_1_1_tcp_congestion_ops.html
- https://www.nsnam.org/docs/release/3.28/doxygen/classns3_1_1_tcp_congestion_ops.html



THANK YOU